
unilasalle RS
50 anos de vidas

Algoritmos e Programação

Aula 10 –Modularização em Python



Dados do Professor

Prof M.e Rafael Gastão – Graduado em Ciência da Computação e Mestre em Dados e IA pela UFRGS.
rafael.ferreira@unilasalle.edu.br

Porque Modularizar?

Porque Modularizar?



Manutenção Ágil

Facilita a identificação e correção de erros. Ao isolar uma tarefa em um bloco, você garante que alterações não quebrem o resto do sistema.



Reuso de Código

Escreva uma vez, use em qualquer lugar. Evita a redundância e reduz drasticamente o tamanho dos arquivos de código fonte.

Introdução a Sub-rotinas

Introdução a Sub-rotinas

Em programação, uma sub-rotina é um bloco de código que realiza uma tarefa específica;

Em vez de escrever o mesmo código várias vezes, nós o "encapsulamos" em uma sub-rotina que pode ser chamada sempre que necessário;

Existem dois tipos principais:

- Procedimentos: Executam uma tarefa, mas não retornam um valor para quem os chamou.
- Funções: Executam uma tarefa e retornam um valor resultante.

Em Python, ambos são definidos com a palavra-chave def.

Funções e Procedimentos em Python

Funções e Procedimentos em Python

Procedimento (Sem retorno)
Usado geralmente para exibir mensagens ou realizar ações de modificação direta.

```
def saudar_aluno(nome):
    #Este é um procedimento que apenas exibe uma saudação.
    print(f"Bem-vindo à aula de Algoritmos, {nome}!")

# Chamada do procedimento
saudar_aluno(“Kai")
```

Funções e Procedimentos em Python

Função (Com retorno)

Utiliza a palavra-chave **return** para enviar um dado de volta ao ponto de chamada.

```
def calcular_media(nota1, nota2):
    #Esta é uma função que retorna a média aritmética.
    media = (nota1 + nota2) / 2
    return media
# Chamada da função e armazenamento do resultado
resultado = calcular_media(8.5, 7.0)
print(f"A média final é: {resultado}")
```

Conceitos de Sub-blocos

Conceitos de Sub-blocos



Indentação

Em Python, os sub-blocos não usam chaves { }, mas sim o recuo de texto (geralmente 4 espaços).



Hierarquia

Todo código identado logo abaixo de uma instrução def pertence àquela sub-rotina.



Clareza

A estrutura forçada garante que o código seja legível por qualquer desenvolvedor da equipe.

Conceitos de Sub-blocos

Um sub-bloco em Python é definido pela identação. Todo código recuado logo abaixo de uma instrução def, if, for ou while pertence àquele bloco.

```
def verificar_aprovacao(media):
    # Início do sub-bloco da função
    if media >= 7.0:
        # Início do sub-bloco do IF
        status = "Aprovado"
    else:
        # Início do sub-bloco do ELSE
        status = "Exame"

    return status
    # Fim do sub-bloco da função
```

Variáveis Locais vs. Globais

Variáveis Locais vs. Globais

A visibilidade de uma variável depende de onde ela foi declarada. Isso é o que chamamos de Escopo.

Variável Local
É declarada dentro de uma função e só existe ali dentro. Quando a função termina, a variável é destruída.

Variável Global
É declarada fora de qualquer função e pode ser acessada por qualquer parte do código.

Variáveis Locais vs. Globais

Exemplo em Python:

```
# Variável Global
disciplina = "Algoritmos e Programação"

def mostrar_detalhes():
    # Variável Local
    professor = "Gastão"
    print(f"Disciplina: {disciplina}") # Acessa a global
    print(f"Professor: {professor}")   # Acessa a local

mostrar_detalhes()

# O código abaixo geraria um erro, pois 'professor' não existe fora da função
# print(professor)
```

Variáveis Locais vs. Globais

Comparativo

Característica	Variável Local	Variável Global
Onde é definida?	Dentro de uma função	Fora de todas as funções
Onde é visível?	Apenas na própria função	Em todo o programa
Tempo de Vida	Curto (Duração da chamada)	Longo (Duração do programa)
Risco de Erros	Baixo (Protegida)	Alto (Pode ser alterada acidentalmente)

Variáveis Locais vs. Globais

A palavra-chave global

Se você precisar alterar o valor de uma variável global dentro de uma função, deve avisar ao Python explicitamente:

```
contador = 0

def incrementar():
    global contador
    contador += 1
    print(f"Contador atual: {contador}")

incrementar()
incrementar()
```

Exemplo Prático

Exemplo Prático

```
def calculadora_simples(a, b, operacao):
    #Exemplo integrando sub-blocos, funções e variáveis locais.
    if operacao == "soma":
        resultado_local = a + b
    elif operacao == "subtracao":
        resultado_local = a - b
    else:
        return "Operação Inválida"

    return resultado_local

# Testando a função
n1 = float(input("Digite o primeiro número: "))
n2 = float(input("Digite o segundo número: "))
op = input("Digite a operação (soma/subtracao): ").lower()

print(f"O resultado é: {calculadora_simples(n1, n2, op)}")
```

Exercicio Prático



Bons Estudos!!!



unilasalle RS 50 anos de vidas